



```
In [26]: import tensorflow as tf
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
        from tensorflow.keras.preprocessing.image import ImageDataGenerator
        import matplotlib.pyplot as plt
```

```
In [27]: train_path = "Plant_Disease_Dataset/train"
        valid_path = "Plant_Disease_Dataset/valid"
        test_path = "Plant_Disease_Dataset/test"
```

```
In [28]: train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=20,
        zoom_range=0.2,
        horizontal_flip=True
    )

    valid_datagen = ImageDataGenerator(rescale=1./255)
    test_datagen = ImageDataGenerator(rescale=1./255)
```

```
In [29]: train_data = train_datagen.flow_from_directory(
        train_path,
        target_size=(128, 128),
        batch_size=32,
        class_mode='categorical'
    )

    valid_data = valid_datagen.flow_from_directory(
        valid_path,
        target_size=(128, 128),
        batch_size=32,
        class_mode='categorical'
    )

    test_data = test_datagen.flow_from_directory(
        test_path,
        target_size=(128, 128),
        batch_size=32,
        class_mode='categorical'
    )
```

Found 18504 images belonging to 10 classes.
Found 4626 images belonging to 10 classes.
Found 0 images belonging to 0 classes.

```
In [30]: model = Sequential()

        # Conv Layer 1
        model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))
        model.add(MaxPooling2D(2,2))

        # Conv Layer 2
        model.add(Conv2D(64, (3,3), activation='relu'))
```

```

model.add(MaxPooling2D(2,2))

# Conv Layer 3
model.add(Conv2D(128, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))

# Flatten
model.add(Flatten())

# Fully Connected
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))

# Output Layer
model.add(Dense(train_data.num_classes, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_9 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_9 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_10 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_10 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_11 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_11 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_3 (Flatten)	(None, 25088)	0
dense_6 (Dense)	(None, 128)	3,211,392
dropout_3 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 10)	1,290

Total params: 3,305,930 (12.61 MB)

Trainable params: 3,305,930 (12.61 MB)

Non-trainable params: 0 (0.00 B)

In [31]: history = model.fit(

```
train_data,  
epochs=3,  
validation_data=valid_data  
)
```

Epoch 1/3

579/579 ————— **698s** 1s/step - accuracy: 0.6051 - loss: 1.1220 - val_accuracy: 0.8042 - val_loss: 0.5185

Epoch 2/3

579/579 ————— **291s** 503ms/step - accuracy: 0.7819 - loss: 0.6007 - val_accuracy: 0.8982 - val_loss: 0.3054

Epoch 3/3

579/579 ————— **250s** 431ms/step - accuracy: 0.8576 - loss: 0.4184 - val_accuracy: 0.9460 - val_loss: 0.1686

```
In [32]: plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
  
plt.title("CNN Accuracy Graph")  
plt.xlabel("Epochs")  
plt.ylabel("Accuracy")  
plt.legend()  
plt.show()
```

